

MODULE 9 · TROUBLESHOOTING

PVC Problems

Pending PVCs, stuck mounts, and wrong claims

A stuck PVC and a stuck Pod look identical from get

Pending and ContainerCreating are symptoms, not diagnoses. The binding path from PVC to running Pod has three separate layers, and each one fails for a different reason and needs a different fix.

Same output, different root cause: a typo'd StorageClass, a PV that's too small, an accessMode mismatch, or a Pod asking for a claim that doesn't exist.

BY THE END YOU CAN

- Tell which layer a stuck PVC is blocked at
- Spot a misleading ProvisioningFailed event
- Diagnose a wrong `claimName` vs a mount failure

Three layers, three different failures



A PVC or Pod can stall at any one of these layers. Find out **which layer** before you touch any YAML — the fix for layer 1 does nothing for a layer 3 problem.

What the status is telling you

SYMPTOM	EVENT / CLUE	ROOT CAUSE
PVC Pending , no PV	storageclass "X" not found	StorageClass typo'd or missing
PVC Pending , PV Available	(often silent — no event)	PV too small, or accessMode/SC mismatch
PVC Pending , default SC	WaitForFirstConsumer	Expected — bind deferred until a Pod consumes it
Pod Pending	persistentvolumeclaim "X" not found	Pod's claimName typo'd
Pod ContainerCreating	MountVolume.SetUp failed	CSI mount/attach failure, or RWO volume already in use

Describe the PVC, then describe the Pod

- **describe the PVC first** — Events show StorageClass and provisioning failures
- **describe the Pod second** — Events show scheduling and mount/attach failures

ORDER MATTERS

A Pod can be perfectly healthy while its PVC is still Pending for an unrelated reason — check the PVC before you chase the Pod.

```
$ kubectl describe pvc NAME -n NS
# Events: ProvisioningFailed / bound

$ kubectl describe pod NAME -n NS
# Events: FailedScheduling / MountVolume
```

The event can lie; the comparison doesn't

A static PV sits Available, its PVC sits Pending. The event says a StorageClass wasn't found — but that name was only a label tying PV and PVC together, not a real dynamic provisioner. The controller tried dynamic provisioning anyway, and *that's* what failed.

The real blocker is silent. Compare capacity and accessModes side by side to find it.

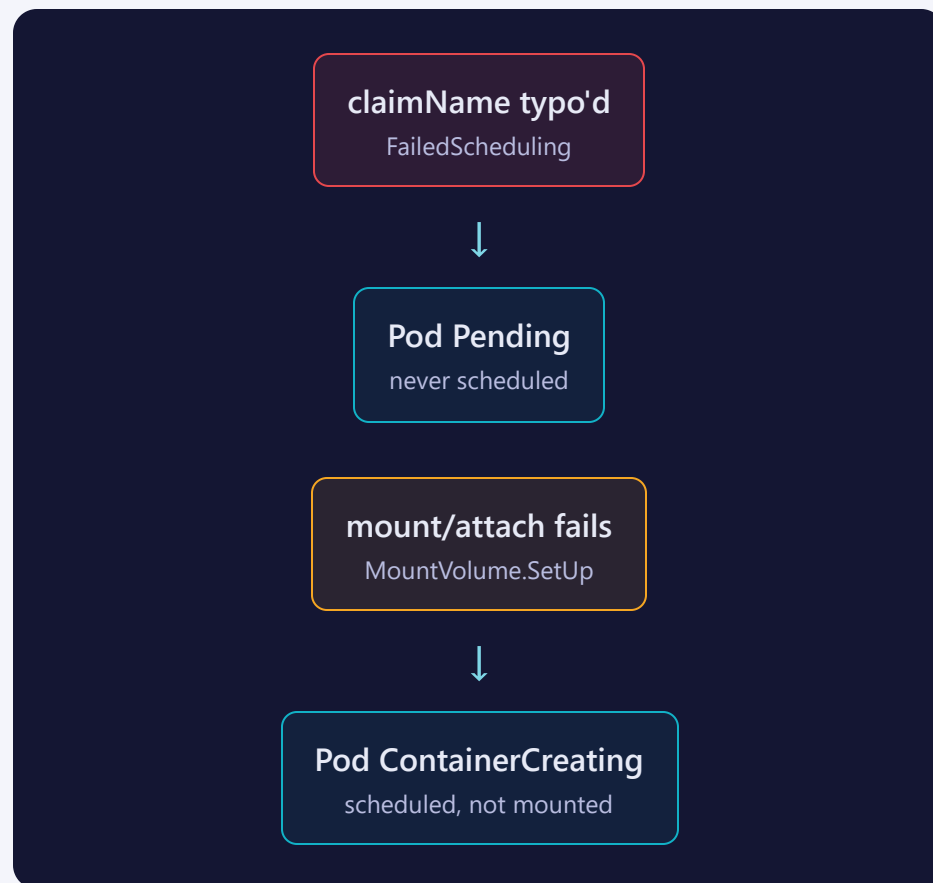
```
PV : Capacity=100Mi  AccessModes=[RW0]
PVC: Request =1Gi    AccessModes=[RW0]

# PV must be >= PVC request
# same SC, same accessMode — size is the
# only mismatch here
```

Wrong claim name vs. failed mount — not the same failure

A Pod referencing a **PVC that doesn't exist** never gets scheduled — the scheduler rejects it outright. Status stays Pending, and the event is a clean FailedScheduling.

A Pod referencing a **PVC that exists but won't mount** — a CSI driver problem, or an RWO volume already attached to a Pod on another node — gets scheduled fine and then hangs in ContainerCreating.



Where people trip

- **Editing storageClassName on an existing PVC.** It's immutable once set — delete and recreate instead.
- **Panicking at WaitForFirstConsumer.** That's a healthy PVC waiting for a Pod, not a broken one.
- **Trusting the first event.** A stale ProvisioningFailed on a static PV is noise — compare capacity/accessMode instead.
- **Forgetting Retain.** A Released PV needs its claimRef cleared by hand before it can rebind.

Commands to keep at hand

```
$ kubectl get pvc,pv -n NS                # the status class
$ kubectl describe pvc NAME -n NS         # StorageClass / provisioning events
$ kubectl describe pod NAME -n NS         # scheduling / mount events
$ kubectl get pv NAME -o jsonpath='...'    # compare capacity vs accessModes
$ kubectl get events -n NS --sort-by=.lastTimestamp
```

Tip: find the layer first — StorageClass, then PV bind, then mount — before you edit anything.

RECAP

PVC triage in one breath

- Same Pending/ContainerCreating symptom, three different layers
- `describe pvc` before `describe pod`
- `WaitForFirstConsumer` is healthy, not broken
- Compare PV vs PVC capacity/accessMode when the event is silent or misleading

HANDS-ON

Now run Lab 9.7
[labs/9.7/](#)

Next: [9.8 — Probe Problems](#)